Teoria dos Grafos e Implementação de Algoritmos em Grafos: Planaridade, Busca em Profundidade, Prim, Kruskal e Dijkstra

Amanda Bialer
Instituto Mauá de Tecnologia — ECM306 Tópicos Avançados em Estruturas de Dados
Prof. Calvetti — 2026

Resumo

Grafos modelam relações entre objetos e aparecem em redes de computadores, mapas rodoviários, circuitos integrados e redes sociais. Este artigo trata de cinco tópicos de Teoria dos Grafos. O primeiro, teórico, é a planaridade: a Fórmula de Euler, os grafos não planares K_5 e $K_{3,3}$, os Teoremas de Kuratowski e Wagner e os testes de planaridade em tempo linear. Os outros quatro são algoritmos implementados em Java com listas de adjacências: a Busca em Profundidade (DFS); os algoritmos de Prim e de Kruskal, que constroem a Árvore Geradora Mínima (AGM); e o algoritmo de Dijkstra, que calcula caminhos mínimos a partir de uma única origem. Para cada algoritmo, o artigo mostra a ideia central, o pseudocódigo, a implementação, um exemplo executado e a análise de complexidade. Prim e Kruskal foram aplicados ao mesmo grafo ponderado e produziram a mesma árvore, de peso total 37. Dijkstra foi aplicado a esse grafo e encontrou as distâncias mínimas a partir do vértice *a*. Os resultados conferem com a literatura de referência (Cormen et al., 2012).

Abstract

Graphs model relationships between objects and appear in computer networks, road maps, integrated circuits and social networks. This paper covers five topics in Graph Theory. The first, theoretical, is planarity: Euler's formula, the non-planar graphs K_5 and $K_{3,3}$, Kuratowski's and Wagner's theorems, and linear-time planarity tests. The other four are algorithms implemented in Java with adjacency lists: Depth-First Search (DFS); Prim's and Kruskal's algorithms, which build a Minimum Spanning Tree (MST); and Dijkstra's single-source shortest-path algorithm. For each algorithm the paper presents the core idea, the pseudocode, the implementation, a worked example and a complexity analysis. Prim's and Kruskal's algorithms were run on the same weighted graph and produced the same tree, of total weight 37. Dijkstra's algorithm was run on that graph and found the shortest distances from vertex *a*. The results match the reference literature (Cormen et al., 2012).

Palavras-chave

Teoria dos Grafos; Planaridade; Busca em Profundidade; Árvore Geradora Mínima; Algoritmo de Prim; Algoritmo de Kruskal; Algoritmo de Dijkstra; Caminhos Mínimos.

Keywords: Graph Theory; Planarity; Depth-First Search; Minimum Spanning Tree; Prim's Algorithm; Kruskal's Algorithm; Dijkstra's Algorithm; Shortest Paths.

1. Introdução

Um grafo $G = (V, E)$ é formado por um conjunto V de vértices e um conjunto E de arestas, que ligam pares de vértices. Em um grafo **não direcionado**, a aresta (u, v) é igual à aresta (v, u). Em um grafo **direcionado**, o par é ordenado. Quando cada aresta tem um número associado por uma função $w : E \to \mathbb{R}$, o grafo é **ponderado**, e esse número é o peso da aresta (Cormen et al., 2012).

Em computador, um grafo costuma ser representado de duas formas. A **matriz de adjacências** ocupa $\Theta(V^2)$ de memória. As **listas de adjacências** ocupam $\Theta(V + E)$ e por isso são preferidas para grafos esparsos, em que $|E|$ é bem menor que $|V|^2$. Todas as implementações deste artigo usam listas de adjacências, como nas atividades práticas anteriores da disciplina.

Muitos problemas práticos se escrevem em termos de grafos:

- percorrer todos os vértices alcançáveis a partir de uma origem (busca);
- ligar todos os pontos de uma rede com o menor custo total (árvore geradora mínima);
- achar a rota mais curta entre dois pontos (caminhos mínimos);
- desenhar uma rede no plano sem que as ligações se cruzem (planaridade).

O artigo está organizado assim. A Seção 2.1 apresenta a teoria da planaridade. A Seção 2.2 trata da Busca em Profundidade. As Seções 2.3 e 2.4 tratam dos algoritmos de Prim e de Kruskal para a Árvore Geradora Mínima. A Seção 2.5 trata do algoritmo de Dijkstra. A Seção 3 apresenta as conclusões.

2. Desenvolvimento do Tema

2.1 Planaridade de Grafos

2.1.1 Definições

Um grafo é **planar** quando pode ser desenhado no plano sem que duas arestas se cruzem, exceto nos vértices em que se encontram. Um desenho desse tipo se chama **imersão plana** (ou grafo

plano). Uma imersão plana divide o plano em regiões conexas chamadas **faces**. Uma delas, a **face externa**, é ilimitada (Bondy; Murty, 2008).

Um grafo pode ser planar mesmo que um desenho específico dele tenha cruzamentos. O K_4 desenhado como um quadrado com as duas diagonais tem um cruzamento. Mas, se uma das diagonais for desenhada por fora do quadrado, o cruzamento desaparece. Ser planar é uma propriedade do grafo, não de um desenho.

2.1.2 Fórmula de Euler

Teorema (Euler). Se G é um grafo plano conexo com V vértices, E arestas e F faces, então:

$$\mathbf{V - E + F = 2}$$

Esboço da prova (indução em E). Se G é uma árvore, então $E = V - 1$ e só existe uma face, a externa. Logo $V - (V - 1) + 1 = 2$. Se G não é uma árvore, ele tem um ciclo. Seja e uma aresta desse ciclo. Ela separa duas faces diferentes, e removê-la junta essas faces em uma só. O grafo $G - e$ continua conexo, com $E - 1$ arestas e $F - 1$ faces. Pela hipótese de indução, $V - (E - 1) + (F - 1) = 2$, que é a mesma igualdade $V - E + F = 2$. ■

Corolário 1. Se G é planar simples com $V \geq 3$, então $\mathbf{E \leq 3V - 6}$. Cada face é limitada por pelo menos 3 arestas, e cada aresta faz fronteira com no máximo 2 faces. Logo $3F \leq 2E$. Substituindo $F = 2 - V + E$ da fórmula de Euler, chega-se a $E \leq 3V - 6$.

Corolário 2. Se G é planar simples, sem triângulos e com $V \geq 3$, então $\mathbf{E \leq 2V - 4}$. A prova é a mesma, mas agora cada face tem pelo menos 4 arestas.

Corolário 3. Todo grafo planar simples tem um vértice de grau ≤ 5. Se todos os vértices tivessem grau ≥ 6, valeria $2E \geq 6V$, o que contradiz o Corolário 1. Esse fato é a base da prova de que todo grafo planar pode ser colorido com 5 cores.

2.1.3 Os grafos K_5 e $K_{3,3}$

Os corolários servem para mostrar que um grafo **não** é planar:

Grafo	V	E	Limite	Conclusão
K_5 (completo, 5 vértices)	5	10	$3 \cdot 5 - 6 = 9$	$10 > 9 \rightarrow$ **não planar**
$K_{3,3}$ (bipartido completo, sem triângulos)	6	9	$2 \cdot 6 - 4 = 8$	$9 > 8 \rightarrow$ **não planar**

O $K_{3,3}$ corresponde ao clássico problema das "três casas e três serviços" (água, luz e gás): não é possível ligar cada casa a cada serviço sem que duas ligações se cruzem.

Os corolários dão apenas condições **necessárias**. Um grafo pode respeitar $E \leq 3V - 6$ e mesmo assim não ser planar. Um exemplo é o $K_{3,3}$ com um vértice extra ligado a ele.

2.1.4 Teoremas de Kuratowski e Wagner

Uma **subdivisão** de um grafo H é obtida trocando arestas de H por caminhos (ou seja, inserindo vértices de grau 2 no meio das arestas).

Teorema de Kuratowski (1930). Um grafo é planar se e somente se não contém subgrafo que seja subdivisão de K_5 ou de $K_{3,3}$.

Teorema de Wagner (1937). Um grafo é planar se e somente se não tem K_5 nem $K_{3,3}$ como **menor**. Um menor é um grafo que se obtém removendo vértices, removendo arestas e contraindo arestas.

Os dois teoremas dizem que K_5 e $K_{3,3}$ são as únicas "obstruções" à planaridade. Quando um grafo não é planar, um desses dois grafos aparece "escondido" dentro dele.

2.1.5 Testes de planaridade

Aplicar Kuratowski diretamente, procurando subdivisões, é caro. Hopcroft e Tarjan (1974) criaram o primeiro algoritmo que testa planaridade em tempo **linear, O(V)**. Ele se baseia em uma busca em profundidade (Seção 2.2). Boyer e Myrvold (2004) criaram depois um algoritmo mais simples, também O(V), que adiciona as arestas uma a uma. Em caso positivo, ele devolve uma imersão plana. Em caso negativo, devolve uma subdivisão de K_5 ou de $K_{3,3}$ que prova que o grafo não é planar. Como um grafo planar tem $E \leq 3V - 6$, qualquer grafo com mais arestas que isso já pode ser rejeitado em O(1).

2.1.6 Exemplo e aplicações

O grafo de cidades da Atividade Hands-on 01 tem 5 vértices: Campinas, São Paulo, Campos do Jordão, São Lourenço e Poços de Caldas. Ele tem $V = 5$ e $E = 7 \leq 3 \cdot 5 - 6 = 9$, e o desenho do enunciado não tem cruzamentos, então o grafo é planar. Pela fórmula de Euler, ele tem $F = 7 - 5 + 2 = 4$ faces. São elas os três triângulos (Campinas–SP–Poços, SP–São Lourenço–Poços e SP–Campos do Jordão–São Lourenço) e a face externa.

A planaridade é importante em várias áreas:

- **projeto de circuitos impressos e VLSI**, em que trilhas que se cruzam exigem camadas extras;
- **desenho de grafos e visualização de redes**;

- **cartografia**: o **Teorema das Quatro Cores** (Appel; Haken, 1977) garante que todo mapa planar pode ser colorido com no máximo 4 cores sem que regiões vizinhas tenham a mesma cor;
- **algoritmos especializados**: muitos problemas que são NP-difíceis em grafos gerais têm algoritmos eficientes quando o grafo é planar.

2.2 Implementação de Busca em Profundidade (DFS)

2.2.1 Ideia

A Busca em Largura (BFS), vista na Atividade 03, explora o grafo em "camadas", na ordem da distância à origem. A **Busca em Profundidade** (*Depth-First Search*, DFS) faz o contrário: segue um caminho o mais longe possível e só volta (*backtracking*) quando o vértice atual não tem mais vizinhos por descobrir (Tarjan, 1972). Na BFS os vértices descobertos ficam em uma **fila**. Na DFS eles ficam em uma **pilha**, que pode ser explícita ou ser a própria pilha de chamadas recursivas.

A DFS usa as mesmas cores da BFS. **Branco** é um vértice ainda não descoberto. **Cinza** é um vértice descoberto que ainda está sendo explorado. **Preto** é um vértice terminado. Além disso, a DFS registra dois tempos para cada vértice u: o tempo de descoberta $u.d$ e o tempo de término $u.f$.

2.2.2 Pseudocódigo (Cormen et al., 2012)

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)

DFS-VISIT(G, u)
1  time = time + 1
2  u.d = time
3  u.color = GRAY
4  for each v ∈ G.Adj[u]
5      if v.color == WHITE
6          v.π = u
7          DFS-VISIT(G, v)
8  u.color = BLACK
9  time = time + 1
10 u.f = time
```

2.2.3 Implementação em Java

Arquivo completo: *BuscaProfundidade.java*.

```
public void dfs() {
    cor = new Cor[V];
    pai = new int[V];
```

```
    d = new int[V];
    f = new int[V];
    for (int u = 0; u < V; u++) {
        cor[u] = Cor.BRANCO;
        pai[u] = -1;
    }
    tempo = 0;
    for (int u = 0; u < V; u++)
        if (cor[u] == Cor.BRANCO)
            visita(u);
}

private void visita(int u) {
    tempo++;
    d[u] = tempo;                  // u acaba de ser descoberto
    cor[u] = Cor.CINZA;
    System.out.print(" " + u);
    for (int v : adj.get(u)) {
        if (cor[v] == Cor.BRANCO) {
            pai[v] = u;
            visita(v);
        }
    }
    cor[u] = Cor.PRETO;            // todos os vizinhos de u foram explorados
    tempo++;
    f[u] = tempo;
}
```

2.2.4 Exemplo

O exemplo usa o mesmo grafo direcionado da Atividade 03 (BFS), com as arestas 0→1, 0→2, 1→2, 1→3, 3→4, 2→3, 4→0, 4→1 e 4→5. A busca começa pelo vértice 0. A saída do programa foi:

```
Ordem DFS: 0 1 2 3 4 5
0: d=1 f=12 pai=-1
1: d=2 f=11 pai=0
2: d=3 f=10 pai=1
3: d=4 f=9 pai=2
4: d=5 f=8 pai=3
5: d=6 f=7 pai=4
```

A BFS da Atividade 03, no mesmo grafo, visitou os vértices na ordem `0 2 1 3 4 5`. A DFS foi direto até o vértice 5 pelo caminho 0→1→2→3→4→5 e só depois voltou. Com os tempos *d* e *f*, as arestas se classificam assim:

Aresta	**Tipo**	**Justificativa**
0→1, 1→2, 2→3, 3→4, 4→5	árvore	ligaram um vértice branco ao vértice que o descobriu
0→2, 1→3	avanço (*forward*)	vão até um descendente que já estava preto
4→0, 4→1	retorno (*back*)	vão até um ancestral que ainda estava cinza

A existência de arestas de retorno prova que o grafo tem **ciclos**, por exemplo 0→1→2→3→4→0. Essa é a base de vários algoritmos derivados da DFS: detecção de ciclos,

ordenação topológica (vértices em ordem decrescente de *f*), componentes fortemente conexas, pontos de articulação e o próprio teste de planaridade de Hopcroft–Tarjan.

2.2.5 Complexidade

Cada vértice é visitado (pintado de cinza) uma única vez, e cada lista de adjacências é percorrida uma única vez. Logo, com listas de adjacências o tempo é **Θ(V + E)**. A memória extra é Θ(V), usada pelos vetores e pela pilha de recursão. Com matriz de adjacências, o tempo sobe para $\Theta(V^2)$.

2.3 Implementação do Algoritmo de Prim

2.3.1 O problema da Árvore Geradora Mínima

Seja $G = (V, E)$ um grafo conexo, não direcionado e ponderado. Uma **árvore geradora** de G é um subconjunto $T \subseteq E$ que liga todos os vértices sem formar ciclos. Toda árvore geradora tem exatamente $V - 1$ arestas. A **Árvore Geradora Mínima** (AGM, ou *Minimum Spanning Tree*, MST) é a árvore geradora de menor peso total $w(T) = \Sigma\, w(u,v)$. O problema aparece, por exemplo, quando se quer ligar cidades por cabos ou estradas com o menor custo total.

Prim e Kruskal são algoritmos **gulosos** baseados na **propriedade do corte**. Tome qualquer divisão dos vértices em dois grupos $(S, V - S)$. A aresta de menor peso entre as que ligam os dois grupos (a "aresta leve") pertence a alguma AGM. Por isso ela é **segura** e pode ser adicionada à árvore em construção (Cormen et al., 2012).

2.3.2 Ideia

O algoritmo foi publicado por Jarník (1930) e redescoberto por Prim (1957). Ele faz **uma única árvore crescer** a partir de uma raiz. Em cada passo, o corte separa os vértices que já estão na árvore dos que ainda não estão, e o algoritmo adiciona a aresta mais leve que atravessa esse corte. Para achar essa aresta rapidamente, usa uma **fila de prioridade** (min-heap).

2.3.3 Pseudocódigo (Cormen et al., 2012)

```
MST-PRIM(G, w, r)
1  for each u ∈ G.V
2      u.key = ∞
3      u.π = NIL
4  r.key = 0
5  Q = G.V
6  while Q ≠ ∅
7      u = EXTRACT-MIN(Q)
8      for each v ∈ G.Adj[u]
9          if v ∈ Q and w(u,v) < v.key
10             v.π = u
11             v.key = w(u,v)
```

2.3.4 Implementação em Java

A implementação abaixo é a versão "preguiçosa" (*lazy*): ela coloca **arestas** na fila de prioridade `PriorityQueue` do Java, em vez de diminuir as chaves dos vértices (`DECREASE-KEY`, operação que a classe não oferece). Quando sai da fila uma aresta que leva a um vértice já incluído, ela é descartada, porque formaria um ciclo. Arquivo completo: *Prim.java*.

```
public ArrayList<Aresta> agm(int raiz) {
    boolean[] naArvore = new boolean[V];
    ArrayList<Aresta> arvore = new ArrayList<>();
    PriorityQueue<Aresta> fila = new PriorityQueue<>((a, b) -> a.peso - b.peso);

    naArvore[raiz] = true;
    fila.addAll(adj.get(raiz));

    while (!fila.isEmpty() && arvore.size() < V - 1) {
        Aresta e = fila.poll();              // aresta leve que cruza o corte
        if (naArvore[e.destino])
            continue;                        // formaria ciclo: descarta
        naArvore[e.destino] = true;
        arvore.add(e);
        for (Aresta a : adj.get(e.destino))
            if (!naArvore[a.destino])
                fila.add(a);
    }
    return arvore;
}
```

2.3.5 Exemplo

O exemplo usa o grafo clássico de Cormen et al. (2012, cap. 23), com 9 vértices (*a* a *i*) e 14 arestas:

Aresta	Peso	Aresta	Peso	Aresta	Peso
a–b	4	c–d	7	e–f	10
a–h	8	c–f	4	f–g	2
b–c	8	c–i	2	g–h	1
b–h	11	d–e	9	g–i	6
		d–f	14	h–i	7

Executando a partir da raiz *a*, o programa adicionou as arestas nesta ordem:

```
a-b (4)
a-h (8)
h-g (1)
g-f (2)
f-c (4)
c-i (2)
c-d (7)
d-e (9)
Peso total: 37
```

No segundo passo, *a–h* e *b–c* tinham o mesmo peso (8), e qualquer uma das duas é segura. A escolha só muda **qual** AGM é encontrada, não o **peso** dela. A AGM é única quando todos os pesos são diferentes.

2.3.6 Complexidade

Estrutura da fila de prioridade	Tempo
Vetor simples (busca linear do mínimo)	$O(V^2)$, bom para grafos densos
Heap binário (versão preguiçosa deste artigo)	O(E log E) = O(E log V)
Heap de Fibonacci (Fredman; Tarjan, 1987)	O(E + V log V)

2.4 Implementação do Algoritmo de Kruskal

2.4.1 Ideia

O algoritmo de Kruskal (1956) também constrói a AGM, mas de outro jeito. Em vez de fazer uma única árvore crescer, ele mantém uma **floresta**, que no começo tem um vértice isolado por árvore. O algoritmo percorre as arestas em **ordem crescente de peso** e aceita uma aresta *(u, v)* apenas se *u* e *v* estão em árvores diferentes. Nesse caso, a aresta une as duas árvores. Se *u* e *v* já estão na mesma árvore, a aresta formaria um ciclo e é descartada.

Para saber rapidamente se dois vértices estão na mesma árvore, o algoritmo usa a estrutura de **conjuntos disjuntos** (*Union-Find*). A operação `FIND-SET(x)` devolve o representante do conjunto de *x*, e `UNION(x, y)` junta dois conjuntos. Com as heurísticas de **compressão de caminho** e **união por rank**, cada operação custa $O(\alpha(V))$ amortizado. Aqui α é a inversa da função de Ackermann, que vale no máximo 4 para qualquer entrada de tamanho realista.

2.4.2 Pseudocódigo (Cormen et al., 2012)

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for each vertex v ∈ G.V
3      MAKE-SET(v)
4  sort the edges of G.E into nondecreasing order by weight w
5  for each edge (u,v) ∈ G.E, taken in nondecreasing order by weight
6      if FIND-SET(u) ≠ FIND-SET(v)
7          A = A ∪ {(u,v)}
8          UNION(u, v)
9  return A
```

2.4.3 Implementação em Java

Arquivo completo: *Kruskal.java*.

```
// Conjuntos disjuntos (Union-Find)
static class ConjuntosDisjuntos {
    int[] pai, rank;

    ConjuntosDisjuntos(int n) {
```

```
        pai = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++)
            pai[i] = i;                         // MAKE-SET
    }

    int find(int x) {                           // FIND-SET com compressão de caminho
        if (pai[x] != x)
            pai[x] = find(pai[x]);
        return pai[x];
    }

    boolean union(int x, int y) {               // UNION por rank
        int rx = find(x), ry = find(y);
        if (rx == ry)
            return false;
        if (rank[rx] < rank[ry])
            pai[rx] = ry;
        else if (rank[rx] > rank[ry])
            pai[ry] = rx;
        else {
            pai[ry] = rx;
            rank[rx]++;
        }
        return true;
    }
}

public ArrayList<Aresta> agm() {
    ArrayList<Aresta> arvore = new ArrayList<>();
    ConjuntosDisjuntos cd = new ConjuntosDisjuntos(V);
    Collections.sort(arestas);

    for (Aresta e : arestas) {
        if (cd.union(e.origem, e.destino))    // aceita só se não formar ciclo
            arvore.add(e);
        if (arvore.size() == V - 1)
            break;
    }
    return arvore;
}
```

2.4.4 Exemplo

O exemplo usa o mesmo grafo da Seção 2.3.5. As arestas são examinadas em ordem crescente de peso:

Ordem	Aresta	Peso	Decisão
1	g–h	1	aceita
2	c–i	2	aceita
3	f–g	2	aceita
4	a–b	4	aceita
5	c–f	4	aceita (une {c,i} com {f,g,h})
6	g–i	6	**rejeitada**: g e i já estão no mesmo conjunto
7	c–d	7	aceita
8	h–i	7	**rejeitada**: formaria ciclo

Ordem	Aresta	Peso	Decisão
9	a–h	8	aceita (une {a,b} ao restante)
10	b–c	8	**rejeitada**: formaria ciclo
11	d–e	9	aceita, e a árvore fica com V − 1 = 8 arestas

Saída do programa:

```
g-h (1)
c-i (2)
f-g (2)
a-b (4)
c-f (4)
c-d (7)
a-h (8)
d-e (9)
Peso total: 37
```

O conjunto de arestas é **o mesmo** encontrado pelo algoritmo de Prim, mas obtido em outra ordem. Prim faz uma única árvore crescer a partir de *a*. Kruskal junta florestas começando pelas arestas mais leves do grafo.

2.4.5 Complexidade

A ordenação das arestas custa O(E log E). As operações de *Union-Find* custam O(E · α(V)) no total. Como $E < V^2$, vale $\log E = O(\log V)$, e o tempo total é **O(E log V)**. Na prática, Kruskal funciona bem em grafos esparsos ou quando as arestas já estão em uma lista. Prim com vetor simples ($O(V^2)$) tende a ser melhor em grafos densos.

2.5 Implementação do Algoritmo de Dijkstra

2.5.1 O problema dos caminhos mínimos de origem única

Seja $G = (V, E)$ um grafo ponderado e s um vértice de origem. O problema é calcular, para todo $v \in V$, o peso do caminho mais leve de s até v, chamado $\delta(s, v)$. A BFS (Atividade 03) resolve esse problema quando todas as arestas pesam 1. O algoritmo de **Dijkstra** (1959) resolve o caso geral, desde que **nenhum peso seja negativo**.

2.5.2 Ideia

O algoritmo usa duas ideias:

1. **Relaxamento.** Para cada vértice v, o algoritmo mantém uma estimativa $v.d$ da distância a partir de s. Relaxar a aresta (u, v) significa testar se passar por u melhora essa estimativa: se $u.d + w(u,v) < v.d$, então $v.d$ recebe $u.d + w(u,v)$ e $v.\pi$ recebe u.

2. **Escolha gulosa.** Entre os vértices ainda não finalizados, o algoritmo sempre pega o de menor *d* e o declara finalizado. Ele então relaxa todas as arestas que saem dele. Como os pesos não são negativos, nenhum caminho que passe por vértices ainda abertos pode ficar mais curto depois, e por isso $u.d = \delta(s, u)$ no momento em que *u* é retirado da fila.

Dijkstra se parece muito com Prim. A diferença está na chave da fila de prioridade: em Prim é o peso da **aresta** que liga *v* à árvore, $w(u, v)$. Em Dijkstra é a distância **acumulada** desde a origem, $u.d + w(u, v)$.

2.5.3 Pseudocódigo (Cormen et al., 2012)

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)      // v.d = ∞, v.π = NIL; s.d = 0
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)

RELAX(u, v, w)
1  if v.d > u.d + w(u,v)
2      v.d = u.d + w(u,v)
3      v.π = u
```

2.5.4 Implementação em Java

Como no algoritmo de Prim, a implementação insere uma nova entrada na `PriorityQueue` sempre que uma distância melhora. As entradas antigas são descartadas ao sair da fila, quando o vértice já está fechado. Arquivo completo: *Dijkstra.java*.

```
public void caminhosMinimos(int s) {
    dist = new int[V];
    pai = new int[V];
    boolean[] fechado = new boolean[V];
    Arrays.fill(dist, Integer.MAX_VALUE);     // INITIALIZE-SINGLE-SOURCE
    Arrays.fill(pai, -1);
    dist[s] = 0;

    // cada entrada da fila é {vértice, distância}
    PriorityQueue<int[]> fila = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    fila.add(new int[]{s, 0});

    while (!fila.isEmpty()) {
        int u = fila.poll()[0];               // EXTRACT-MIN
        if (fechado[u])
            continue;
        fechado[u] = true;
        for (Aresta e : adj.get(u)) {         // RELAX(u, v, w)
            int v = e.destino;
            if (!fechado[v] && dist[u] + e.peso < dist[v]) {
                dist[v] = dist[u] + e.peso;
                pai[v] = u;
                fila.add(new int[]{v, dist[v]});
            }
        }
    }
}
```

```
public String caminho(int v, String nomes) {
    if (pai[v] == -1)
        return "" + nomes.charAt(v);
    return caminho(pai[v], nomes) + " -> " + nomes.charAt(v);
}
```

2.5.5 Exemplo

O exemplo usa o mesmo grafo ponderado das Seções 2.3 e 2.4, tratado como não direcionado, com origem em *a*. Os vértices foram finalizados na ordem *a (0), b (4), h (8), g (9), f (11), c (12), i (14), d (19), e (21)*. Alguns relaxamentos importantes:

- Ao fechar *b* (d = 4), o vértice *c* recebe *4 + 8 = 12*, e *h* continua com 8 (porque *4 + 11 = 15* é pior).
- Ao fechar *f* (d = 11), a estimativa *c.d = 11 + 4 = 15* **não** melhora o valor 12 que *c* já tinha. Já *e* recebe *11 + 10 = 21*.
- Ao fechar *c* (d = 12), o vértice *i* melhora de 15 (via *h*) para *12 + 2 = 14*.

Saída do programa:

```
a: dist=0  caminho: a
b: dist=4  caminho: a -> b
c: dist=12  caminho: a -> b -> c
d: dist=19  caminho: a -> b -> c -> d
e: dist=21  caminho: a -> h -> g -> f -> e
f: dist=11  caminho: a -> h -> g -> f
g: dist=9  caminho: a -> h -> g
h: dist=8  caminho: a -> h
i: dist=14  caminho: a -> b -> c -> i
```

A **árvore de caminhos mínimos** formada pelos ponteiros π tem peso total 4 + 8 + 8 + 1 + 2 + 2 + 7 + 10 = 42. Esse valor é maior que o da AGM (37). Os dois algoritmos otimizam objetivos diferentes. A AGM minimiza o custo **total** de ligar todos os vértices. A árvore de Dijkstra minimiza a distância de **cada** vértice até a origem.

2.5.6 Complexidade e limitações

Estrutura da fila de prioridade	Tempo
Vetor simples	$O(V^2)$
Heap binário (versão deste artigo)	$O((V + E) \log V)$
Heap de Fibonacci	$O(E + V \log V)$

Com pesos negativos, Dijkstra pode errar, porque um vértice já fechado poderia depois ser alcançado por um caminho mais barato. Nesse caso é preciso usar o algoritmo de **Bellman–Ford**, que roda em $O(V \cdot E)$ e também detecta ciclos de peso negativo. O cálculo `dist[u] + e.peso` não estoura `Integer.MAX_VALUE`, porque um vértice só sai da fila depois de receber uma distância finita.

3. Conclusões / Considerações Finais

Este artigo apresentou cinco tópicos centrais da Teoria dos Grafos. Na parte teórica, a **Fórmula de Euler** ($V - E + F = 2$) leva a condições necessárias simples para a planaridade, como $E \leq 3V - 6$. Essas condições bastam para provar que K_5 e $K_{3,3}$ não são planares. Os teoremas de **Kuratowski** e **Wagner** mostram que esses dois grafos são as únicas obstruções à planaridade, e algoritmos baseados em DFS testam planaridade em tempo linear.

Na parte prática, quatro algoritmos foram implementados em Java com listas de adjacências e testados:

Algoritmo	**Problema**	**Estratégia**	**Complexidade (implementação)**
DFS	percurso / estrutura do grafo	recursão (pilha)	$\Theta(V + E)$
Prim	árvore geradora mínima	guloso, uma árvore cresce	$O(E \log V)$
Kruskal	árvore geradora mínima	guloso, floresta + Union-Find	$O(E \log V)$
Dijkstra	caminhos mínimos de origem única	guloso + relaxamento	$O((V + E) \log V)$

Os experimentos confirmaram o comportamento esperado. A DFS encontrou arestas de retorno e com isso revelou os ciclos do grafo da Atividade 03. Prim e Kruskal chegaram à mesma AGM, de peso 37, por caminhos diferentes. Dijkstra encontrou as distâncias mínimas a partir de *a*. A comparação entre a AGM (peso 37) e a árvore de caminhos mínimos (peso 42) mostra que "ligar tudo com o menor custo" e "chegar a cada ponto pelo menor caminho" são problemas diferentes.

Os algoritmos também se relacionam entre si. A DFS é a base do teste de planaridade de Hopcroft–Tarjan. Prim e Dijkstra têm a mesma estrutura e diferem só na chave da fila de prioridade. A escolha da estrutura de dados (vetor, heap binário ou heap de Fibonacci; lista ou matriz de adjacências) muda a complexidade final e deve levar em conta a densidade do grafo. Possíveis continuações deste estudo são o algoritmo de Bellman–Ford para pesos negativos, o algoritmo A* para buscas guiadas por heurística e a implementação de um teste completo de planaridade.

Referências Bibliográficas

APPEL, K.; HAKEN, W. Every planar map is four colorable. Part I: Discharging. **Illinois Journal of Mathematics**, v. 21, n. 3, p. 429–490, 1977.

BONDY, J. A.; MURTY, U. S. R. **Graph Theory**. London: Springer, 2008. (Graduate Texts in Mathematics, 244).

BOYER, J. M.; MYRVOLD, W. J. On the cutting edge: simplified O(n) planarity by edge addition. **Journal of Graph Algorithms and Applications**, v. 8, n. 3, p. 241–273, 2004.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

DIJKSTRA, E. W. A note on two problems in connexion with graphs. **Numerische Mathematik**, v. 1, p. 269–271, 1959.

FEOFILOFF, P.; KOHAYAKAWA, Y.; WAKABAYASHI, Y. **Uma introdução sucinta à teoria dos grafos**. São Paulo: IME-USP, 2011.

FREDMAN, M. L.; TARJAN, R. E. Fibonacci heaps and their uses in improved network optimization algorithms. **Journal of the ACM**, v. 34, n. 3, p. 596–615, 1987.

HOPCROFT, J.; TARJAN, R. Efficient planarity testing. **Journal of the ACM**, v. 21, n. 4, p. 549–568, 1974.

JARNÍK, V. O jistém problému minimálním. **Práce Moravské Přírodovědecké Společnosti**, v. 6, p. 57–63, 1930.

KRUSKAL, J. B. On the shortest spanning subtree of a graph and the traveling salesman problem. **Proceedings of the American Mathematical Society**, v. 7, n. 1, p. 48–50, 1956.

KURATOWSKI, K. Sur le problème des courbes gauches en topologie. **Fundamenta Mathematicae**, v. 15, p. 271–283, 1930.

PRIM, R. C. Shortest connection networks and some generalizations. **Bell System Technical Journal**, v. 36, n. 6, p. 1389–1401, 1957.

SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Boston: Addison-Wesley, 2011.

TARJAN, R. Depth-first search and linear graph algorithms. **SIAM Journal on Computing**, v. 1, n. 2, p. 146–160, 1972.

WAGNER, K. Über eine Eigenschaft der ebenen Komplexe. **Mathematische Annalen**, v. 114, p. 570–590, 1937.